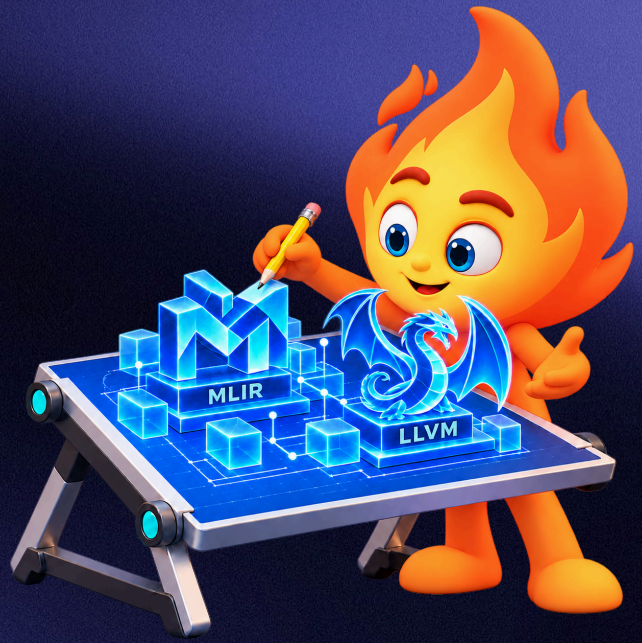
Mojo 🔥 Compiler Architecture Overview



Billy Zhu · Weiwei Chen
Modular, a Qualcomm Company
billyzhu@modular.com, weiwei.chen@modular.com
Mojo Workshop - LLVM Dev Meeting 2026

Mojo as “LLVM Layer” for GenAI

LLVM

C · C++ · Rust · Swift · Fortran

↓

LLVM IR 

One middle end

↓

x86 · ARM · RISC-V · PowerPC · NVPTX · AMDGPU

Mojo

frameworks · graph compilers · serving

↓

Mojo 🔥

Kernels + hardware abstractions

↓

NVIDIA · AMD · Apple · Qualcomm · TPU · Trainium

[Why ML Needs a New Programming Language with Chris Lattner by Jane Street]

Mojo compiler architecture

Mojo Parser 🔥

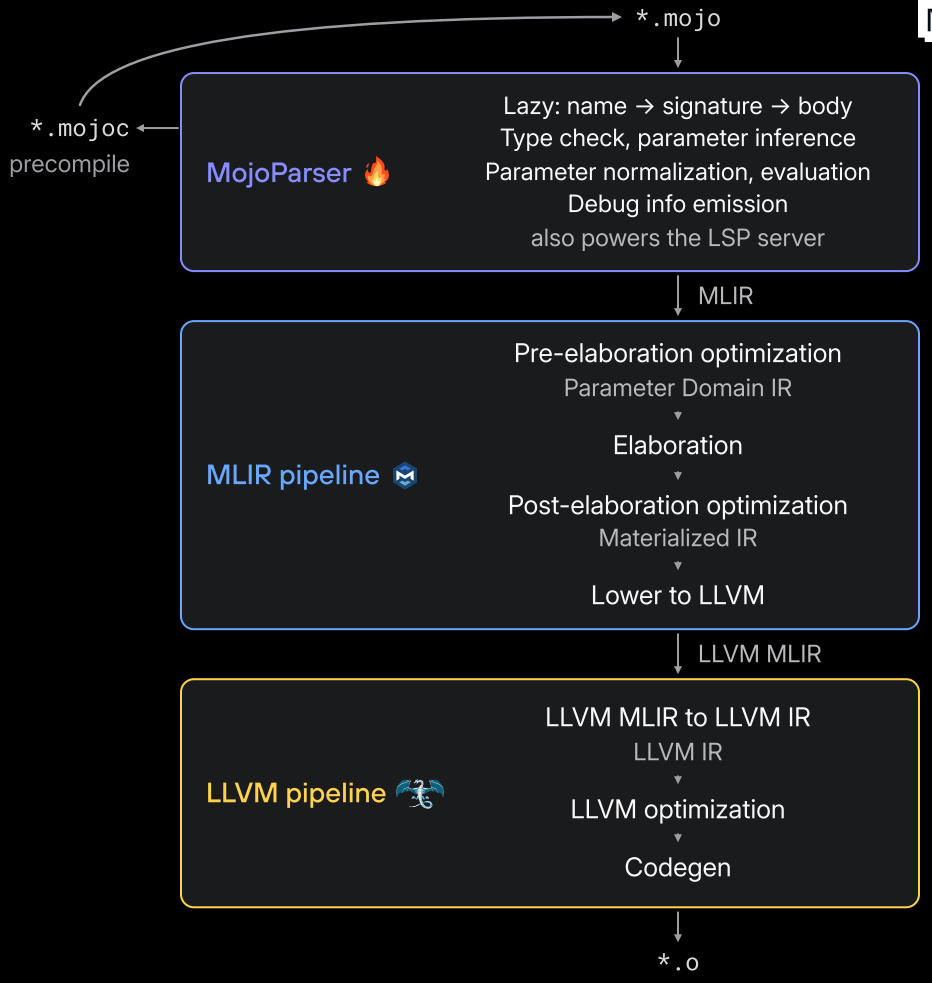
Emits parametric MLIR directly (no AST), lazy resolution, checked generics, parameter normalization, ...

KGENCompiler MLIR layer

Check lifetimes, elaboration, IR optimizations, ...

ObjectCompiler LLVM layer

Lower MLIR to LLVM, LLVM optimization and code generation, ...



Mojo compiler built with both MLIR and LLVM

Mojo's MLIR dialects

LIT	parser dialect, close to source
KGEN	parametric IR foundation, "kernel generator"
HLCF	structured control flow
POP	parametric low-level operations
CO	co-routine support
interp	interpreter support

Upstream MLIR, used sparingly: index and LLVM, plus NVVM and ROCDL for devices. No arith, vector, affine, memref or linalg.

MLIR framework: more modern and efficient

Flexible IR abstraction levels · built-in parallelization for passes · debugging and IR serialization support

Customized LLVM pipeline

- ☐ GVN · load/store optimization · LSR · scalar optimization · target codegen · inliner
- ⊘ Vectorizer · loop unroller · coroutine passes · most inter-procedural passes

Parallelized LLVM compilation

1. Split subgraphs for optimization, call graphs intact
2. Split functions for parallel codegen
3. Relink in memory with MCLink

LLVM share of compile time

~~60–80%~~ → 20–30%

What we've learned from building Mojo's optimization pipeline

Compiler Overview

01 Mojo Parser

How the parser is architected

02 Mojo Elaborator

More on the elaborator, beyond the interpreter

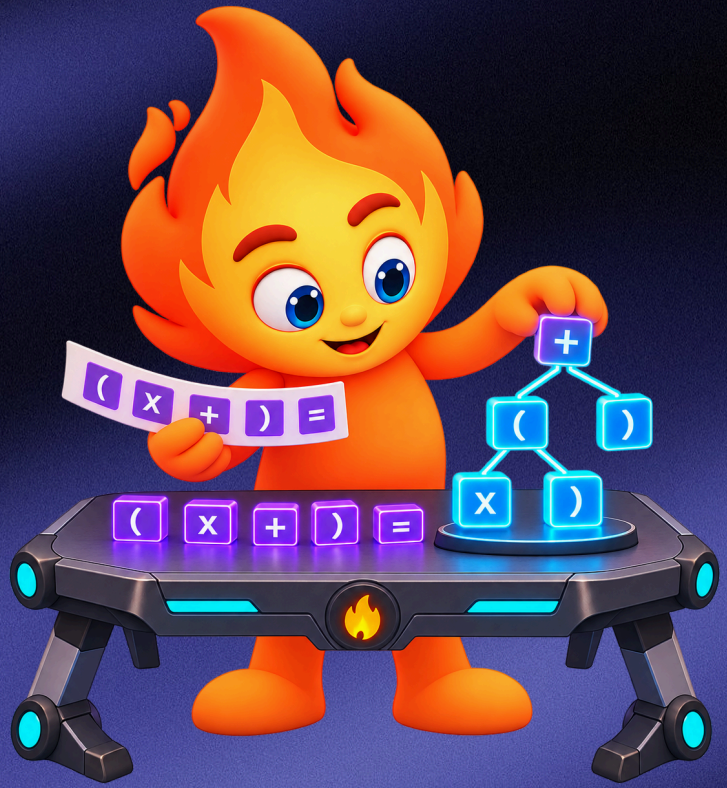
03 Plugin Architecture

For adding new hardware backends

Past talks

EuroLLVM 2024	Mojo debugging
LLVM Dev Meeting 2024	Overall compilation flow
EuroLLVM 2025	LLVM backend with MCLink
LLVM Dev Meeting 2025	Parametric IR representation
EuroLLVM 2026	Attr-based expression system
EuroLLVM 2026	Compile-time interpreter in MLIR

Mojo Parser



Parametric IR in MLIR

Source → MLIR, no AST

Checked generics → parametric IR

REF ↗ Building Modern Language Frontends with MLIR: Lessons from Mojo's Compile-Time Meta-Programming

LLVM Dev Meeting 2025

```
lit.fn @simd_sum
  <size: @Int, dtype: @DType>
  (%value: @SIMD<size, dtype>)
    -> @SIMD<1, dtype> {
  ...
  kgen.param.for iter in ... {
    ...
    kgen.param.call @other_fn<size>()
    ...
    kgen.param.if iter {
      ...
      my.param.op { dt = dtype }
      ...
    }
    kgen.deferred {
      name = "hw.specific",
      attrs = {...} }
  }
  %c = kgen.param.constant = <size>
  ...
}
```

The running example Mojo source to parser output

array.mojo

```
struct Array[T: Copyable, size: Int]:
    ...

def double[n: Int](
    a: Array[Int, n]
) -> Array[Int, n * 2]:
    ...
```

parser output (LIT dialect)

```
lit.struct.decl @Array<T: !Copyable, size: !Int> {
  ...
}
lit.fn @double<n: !Int>(
    %a: !lit.ref<!lit.struct<
        #Array <:!Copyable !Int, :!Int n>>, ...>,
    %__result__: !lit.ref<!lit.struct<
        #Array <:!Copyable !Int, :!Int mul(n, 2)>>,
        ...> byref_result
) -> !kgen.none
```

Parameter list Parameter def ↔ use Parameter expression

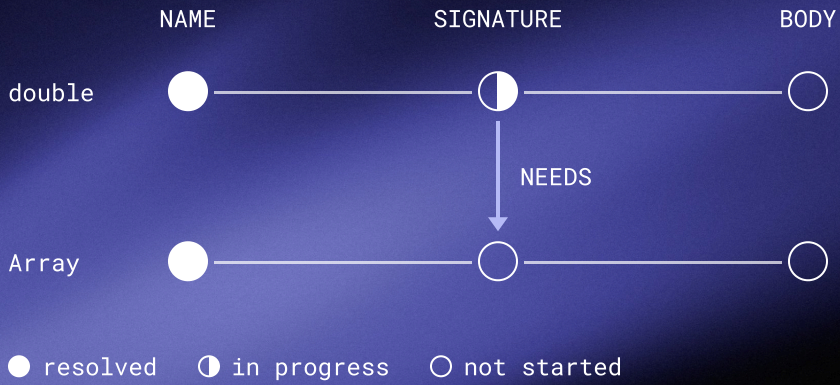
Lazy resolution

Name
↓
Signature
↓
Body

array.mojo

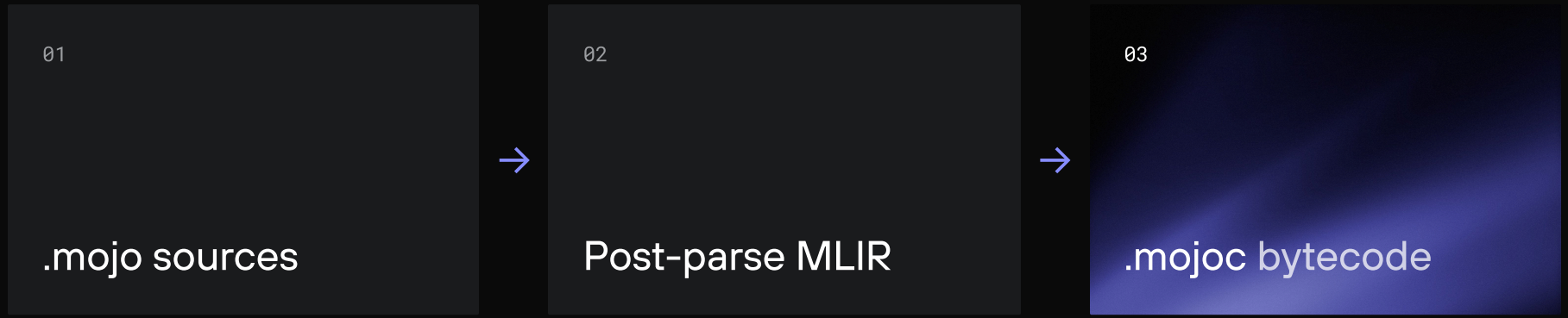
```
struct Array[T: Copyable, size: Int]:
    ...

def double[n: Int](
    a: Array[Int, n]
) -> Array[Int, n * 2]:
    ...
```



Packages are MLIR bytecode. Import a .mojoc in place of the sources.

MOJO PRECOMPILE



Full post-parse, parametric IR, including bodies.

Lazy bytecode parsing

mlir::BytecodeReader

```
// Import a .mojoc: read only the top level
reader = std::make_unique<mlir::BytecodeReader>(
    buffer, config, /*lazyLoad=*/true, sourceMgr);
reader->readTopLevel(block);

// Resolving a declaration materializes it
if (reader->isMaterializable(op))
  reader->materialize(op);

// On teardown, drop whatever nobody asked for
reader->finalize([](Operation *) { return false; });
```

pkg.mojoc

```
lit.struct.decl @Array<T: !Copyable, size: !Int> {

    UNMATERIALIZED BYTECODE

}
```

Same resolvedness progression

Only needed ops are materialized

Type-checking in MLIR

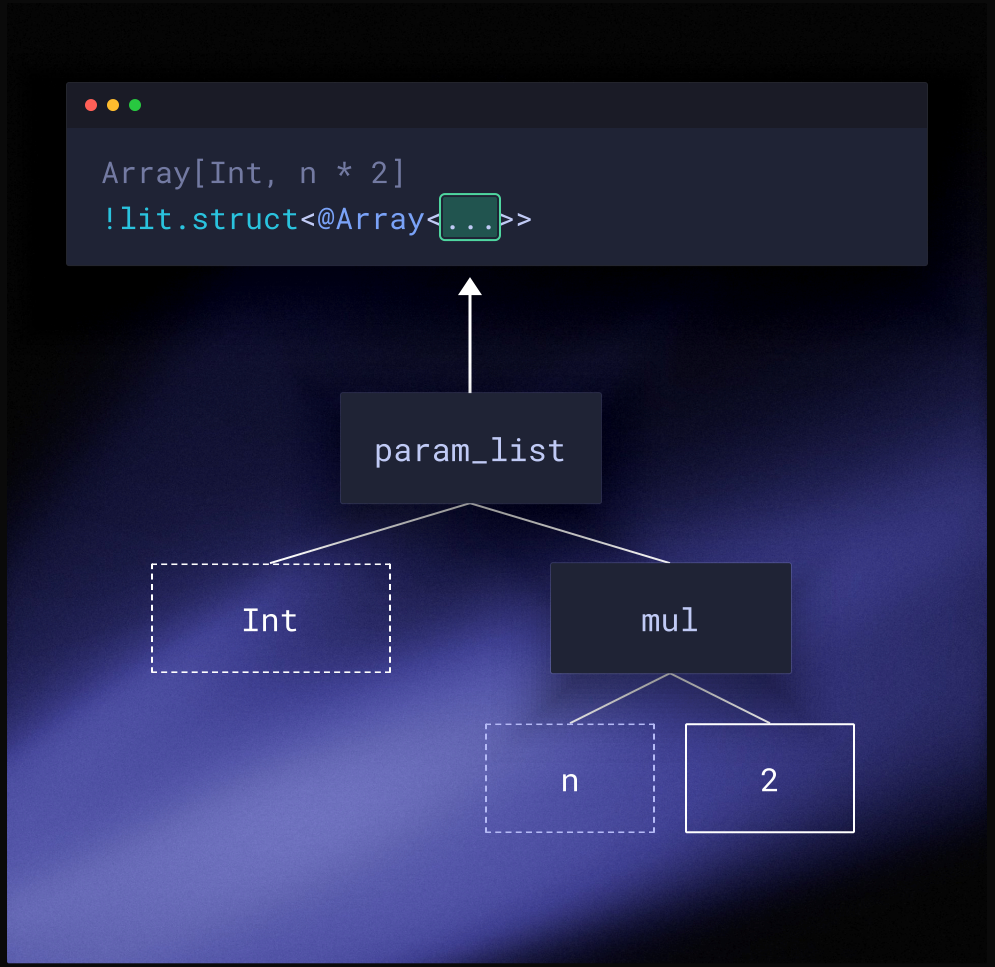


Checked generics

Dependent types

Parameters are attributes.

Compile-time expressions are uniqued attribute trees.



Operators are inner nodes

Constants and parameter refs are leaves

REF ↗ Mojo's Attribute-Based Expression System

Normalizing parameters

```
def double[n: Int](
    a: Array[Int, n]
) -> Array[Int, n * 2]: ...

def take[n: Int](
    a: Array[Int, n + n]
): ...

def caller[n: Int](a: Array[Int, n]):
    take[n](double(a))

# both normalize to mul(n, 2)
```

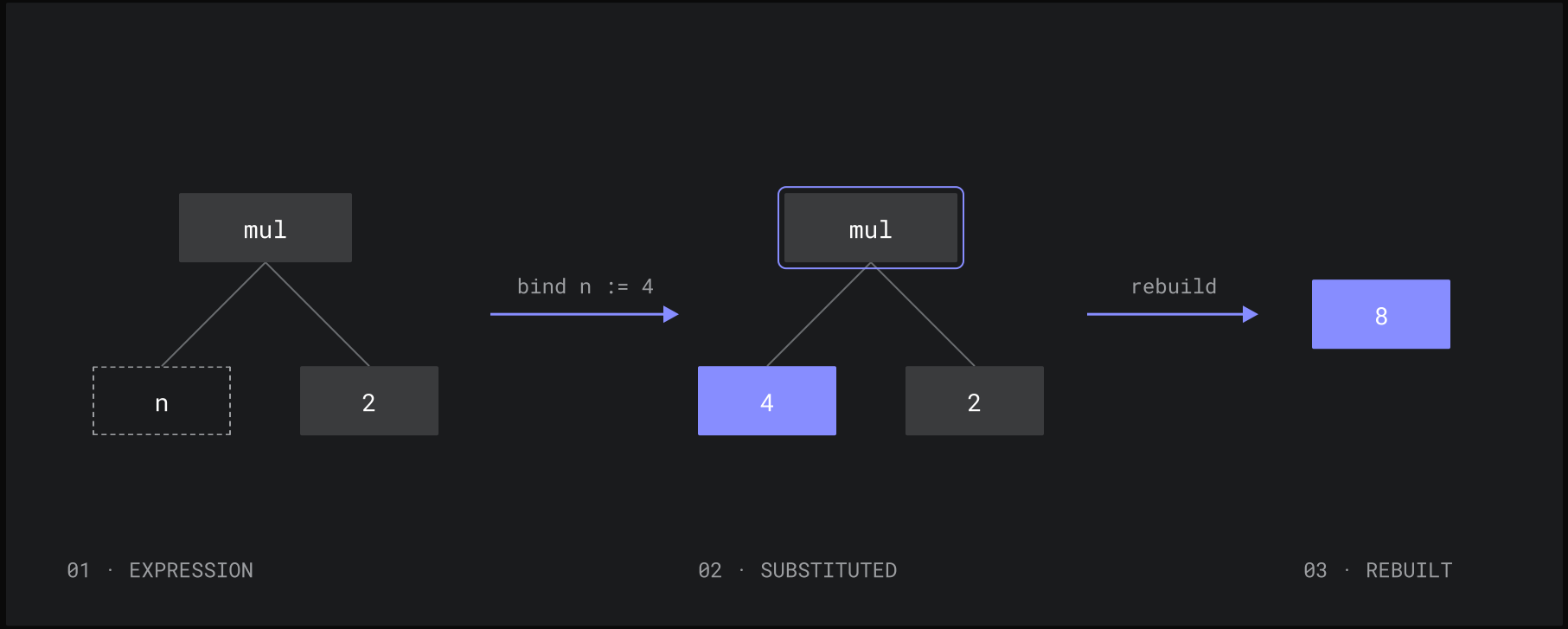
2 + 2 → 4

y + x → x + y

n + n → n * 2

ParameterEvaluator

Substitute, then rebuild bottom-up



Walk and rebuild

with MLIR sub-element APIs

ParameterEvaluator (pseudocode)

```
replace(attr):                  // cached
  if attr is a reference:
    return binding(attr)

  newAttrs = [], newTypes = []
  attr.walkImmediateSubElements(
      a -> newAttrs.push(replace(a)),
      t -> newTypes.push(replace(t)))

  if nothing changed:
    return attr                 // no rebuild
  return attr.replaceImmediateSubElements(
      newAttrs, newTypes)       // re-normalized
```

`walkImmediateSubElements`

Visit each child

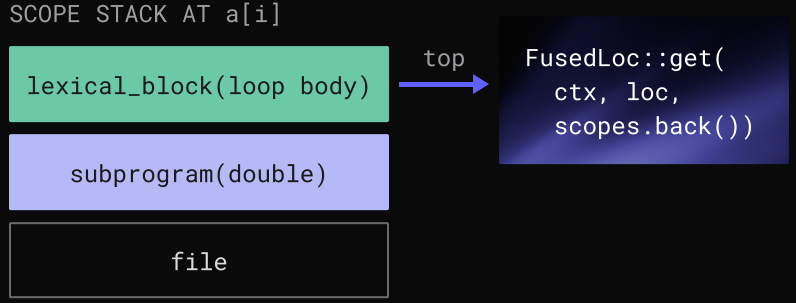
`replaceImmediateSubElements`

Rebuild if changed

`attribute uniquing`

Cache results by pointer

Debug info fused
on locations



A scope stack during resolution

Every op's location carries the innermost scope

example.mojo

```
def double[n: Int](a: Array[Int, n]) -> Array[Int, n * 2]:
    var result = Array[Int, n * 2]()
    for i in range(n):
        result[i] = a[i]   # lexical block starts here
        result[i + n] = a[i]
    return result^
```

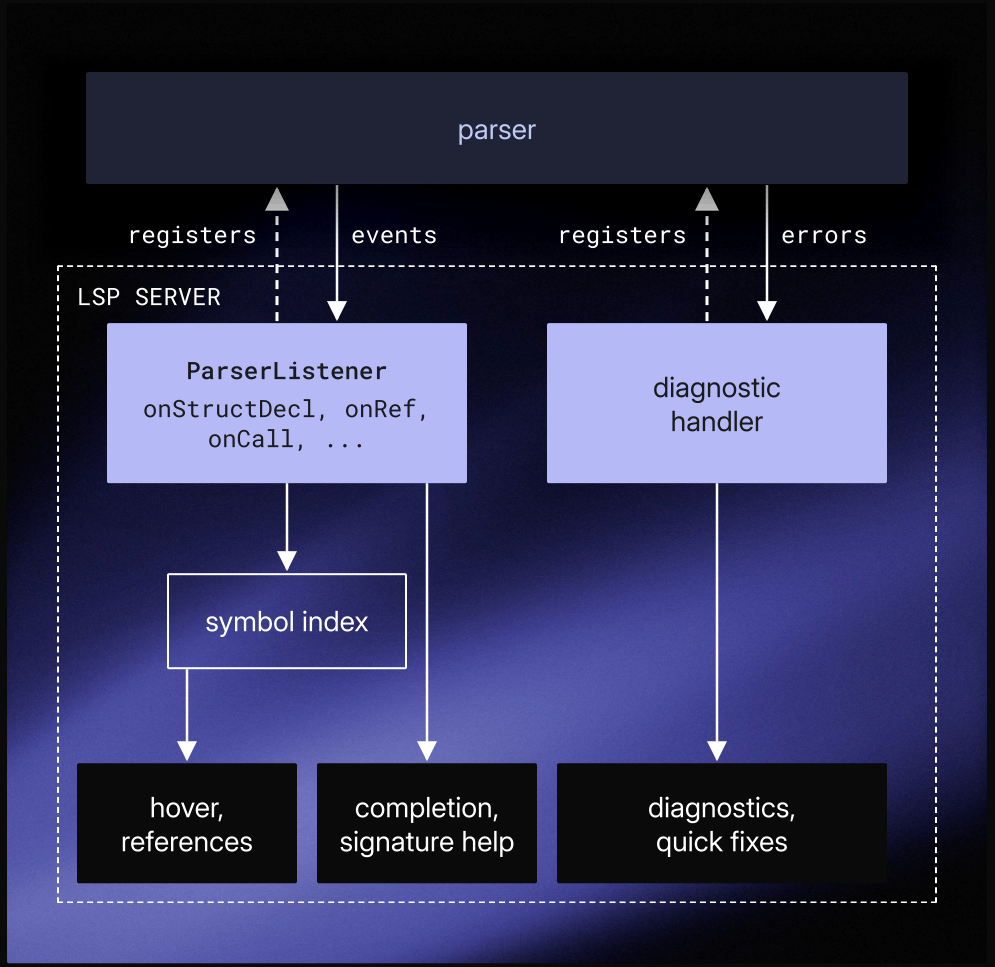
parser output

```
#file = #debuginfo.file<"example.mojo" in "">
#subprogram = #debuginfo.subprogram<
    scope = #file, sourceName = #double_name,
    file = #file, line = 1, scopeLine = 1, ...>
#lexical_block = #debuginfo.lexical_block<
    scope = #subprogram,   // its parent
    file = #file, line = 4, column = 9>

%3 = lit.call @Array::@"__getitem__(...)"(...)
    loc(fused<#lexical_block>["example.mojo":4:22])
```

REF ↗ Mojo Debugging: Extending MLIR and LLDB

LSP drives the same parser



Same resolver, same type checking

Imports stay lazy

Mojo Elaborator



Elaborating Parametric Mojo code

The elaborator concretizes parametric IR by replacing symbolic parameters with concrete types and values, some of which are evaluated by the interpreter.

Mojo code

```
def add[p: Int](arg: Int) -> Int:
    return p + arg

def f(arg: Int) -> Int:    # arg = 3
    comptime c = add[1](2) # c = 1 + 2 = 3
    v1 = add[1](arg)       # v1 = 1 + 3 = 4
    v2 = add[c](arg)       # v2 = c + 3 = 6
    return v1 + v2         # v1 + v2 = 10

def main():
    comptime v = f(3)      # f(3) = 10
    a = add[1](1)          # a = 2
    print(v + a)           # 10 + 2 = 12
```

Parametric IR

```
kgen.generator @print(%arg: index){
   kgen.return
}
kgen.generator @add<p>(%arg: index) -> index {
 %0 = kgen.param.constant = <p>                    // def param p
 %1 = index.add %0, %arg                           // use param p
 kgen.return %1: index
}
kgen.generator @f(%arg: index) -> index  {
 kgen.param.apply c = [(index) -> index: @add<1>](2) // def param c
 %1 = kgen.call @add<1>(%arg) : (index) -> index
 %2 = kgen.call @add<c>(%arg) : (index) -> index     // use param c
 %3 = index.add %1, %2
 kgen.return %3: index
}
kgen.generator export @main() {
 kgen.param.apply v = [(index) -> index: @f](3)      // def param v
 %0 = kgen.param.constant = <v>                      // use param v
 %index1 = kgen.param.constant = <1>
 %1 = kgen.call @add<1>(%index1) : (index) -> index
 %2 = index.add %0, %1
 kgen.call @print(%2): (index) -> ()
 kgen.return
}
```

Concrete IR

```
kgen.func @print(%arg0: index) {
 kgen.return
}
kgen.func @"add,p=1"(%arg0: index) -> index {
 %index1 = kgen.param.constant = <1> // c = 1
 %0 = index.add %index1, %arg0
 kgen.return %0 : index
}
kgen.func @"add,p=3"(%arg0: index) -> index {
 %index3 = kgen.param.constant = <3> // c = 3
 %0 = index.add %index3, %arg0
 kgen.return %0 : index
}
kgen.func @f(%arg0: index) -> index {
 %0 = kgen.call @"add,p=1"(%arg0) : (index) -> index // c = 1
 %1 = kgen.call @"add,p=3"(%arg0) : (index) -> index // c = 3, interpreted!
 %2 = index.add %0, %1
 kgen.return %2 : index
}
kgen.func export @main() {
 %index10 = kgen.param.constant = <10> // v = 10, interpreted!
 %index1 = kgen.param.constant = <1>
 %0 = kgen.call @"add,p=1"(%index1) : (index) -> index
 %1 = index.add %index10, %0
 kgen.call @print(%1) : (index) -> ()
 kgen.return
}
```

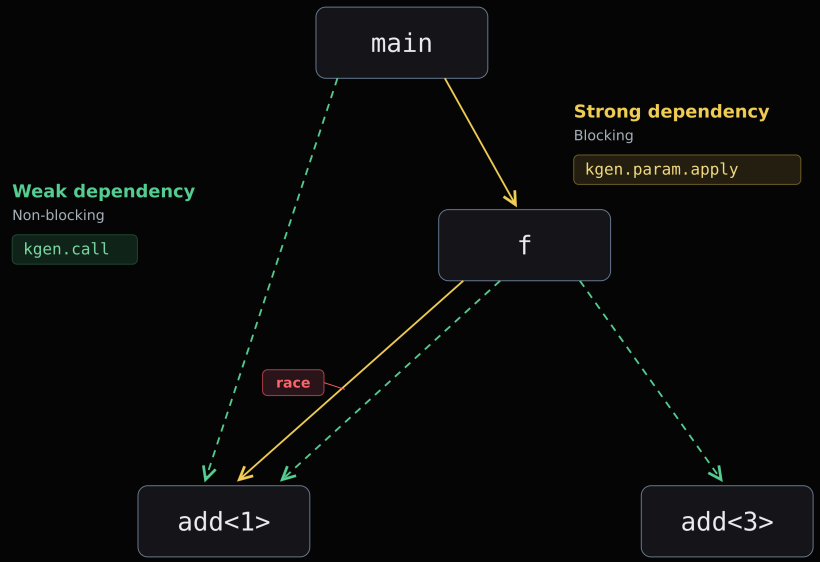
Elaboration dependencies

```
def add[p: Int](arg: Int) -> Int:
    return p + arg

def f(arg: Int) -> Int:    # arg = 3
    comptime c = add[1](2) # c = 1 + 2 = 3
    v1 = add[1](arg)       # v1 = 1 + 3 = 4
    v2 = add[c](arg)       # v2 = c + 3 = 6
    return v1 + v2         # v1 + v2 = 10

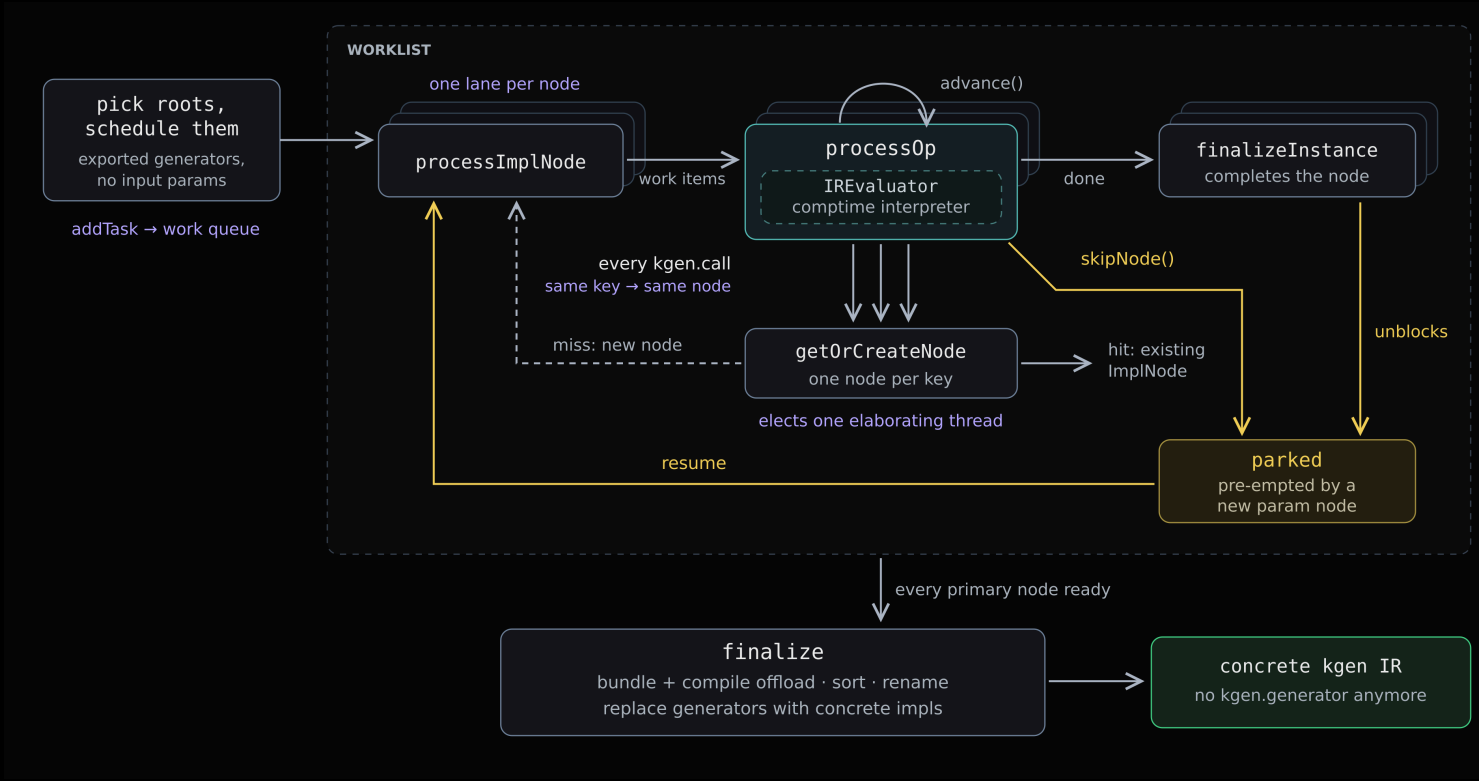
def main():
    comptime v = f(3)      # f(3) = 10
    a = add[1](1)          # a = 2
    print(v + a)           # 10 + 2 = 12
```

- Blocking dependency (comptime)
- Non-blocking dependency (runtime)



- Elaborating each kgen.generator in parallel.
- Elaborating parametric operations in each generator into concrete ones with blocking and non-blocking dependencies.
- Elaboration is marked as done till all dependencies are done.

Elaborator workflow

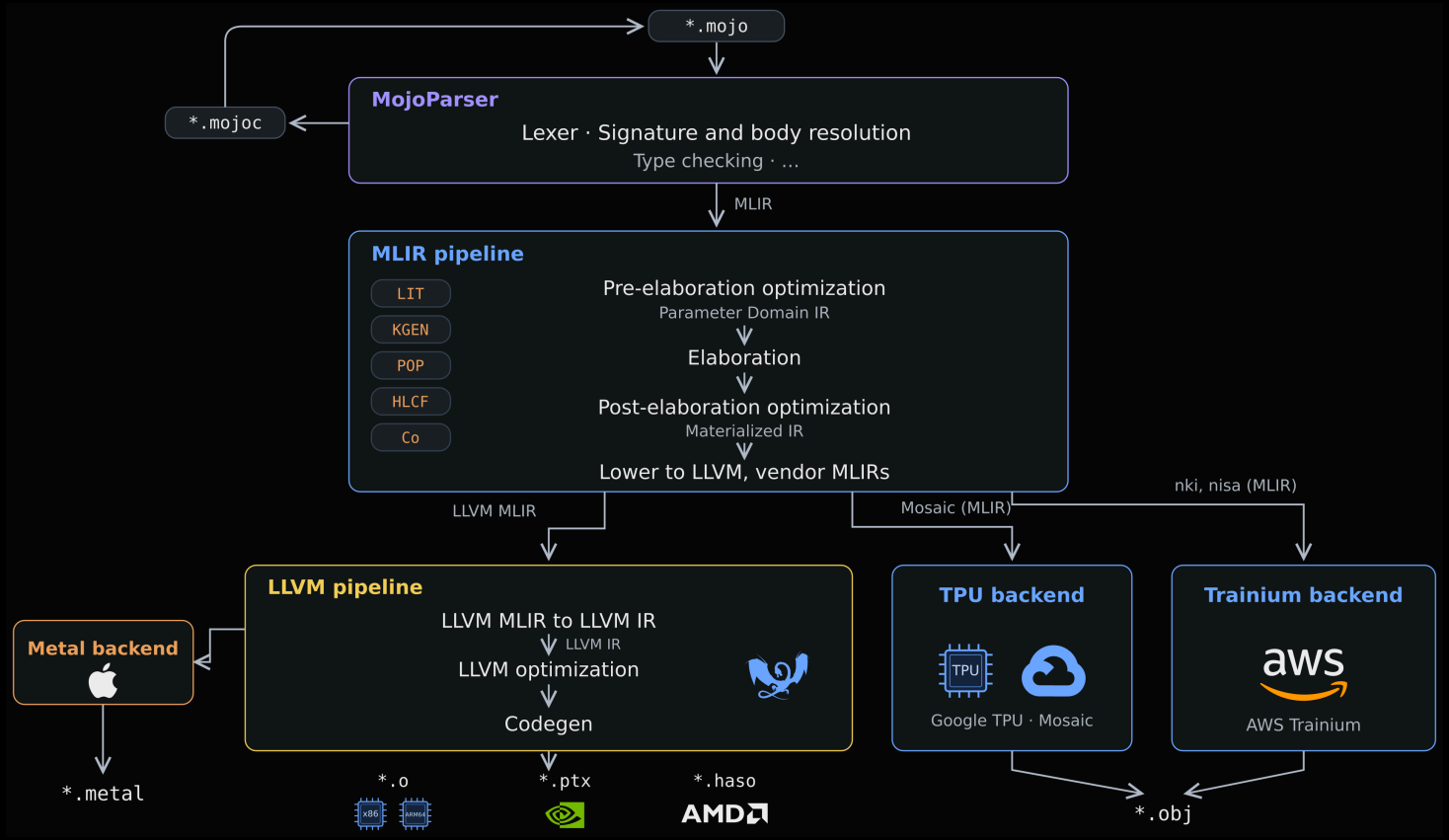


Parallelized, concretize before interpreting, with in-memory interpreter cache.

Mojo Backend Plugins



Mojo Compilation to Different Backends



Mojo Offload Compilation

```mojo
from std.math import ceildiv
from std.sys import has_accelerator

from layout import TileTensor, row_major
from max.gpu import global_idx
from max.gpu.host import DeviceContext

comptime dtype = DType.float32
comptime WIDTH = 10
comptime BLOCK = 8
comptime layout = row_major[WIDTH]()

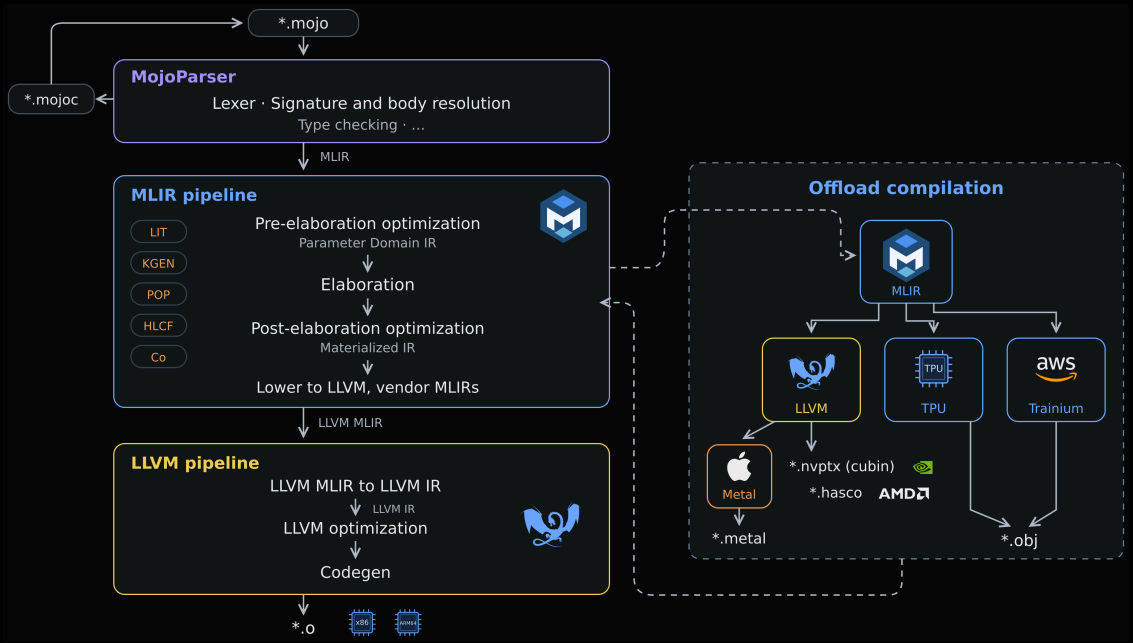
def vector_addition(                                    # GPU kernel
    lhs: TileTensor[dtype, type_of(layout), ImmutAnyOrigin],
    rhs: TileTensor[dtype, type_of(layout), ImmutAnyOrigin],
    output: TileTensor[dtype, type_of(layout), MutAnyOrigin],
    size: Int32,
):
    var tid = global_idx.x
    if tid < Int(size):
        output[tid] = lhs[tid] + rhs[tid]

def main() raises:                                      # CPU host code
    comptime assert has_accelerator(), "needs a GPU"
    var ctx = DeviceContext()

    var lhs_buf = ctx.enqueue_create_buffer[dtype](WIDTH)
    var rhs_buf = ctx.enqueue_create_buffer[dtype](WIDTH)
    var out_buf = ctx.enqueue_create_buffer[dtype](WIDTH)
    lhs_buf.enqueue_fill(1.0)
    rhs_buf.enqueue_fill(2.0)

    var lhs = TileTensor(lhs_buf, layout)
    var rhs = TileTensor(rhs_buf, layout)
    var output = TileTensor(out_buf, layout)
    ctx.enqueue_function[vector_addition](
        lhs.as_imm(),
        rhs.as_imm(),
        output,
        Int32(WIDTH),
        grid_dim=ceildiv(WIDTH, BLOCK),
        block_dim=BLOCK,
    )

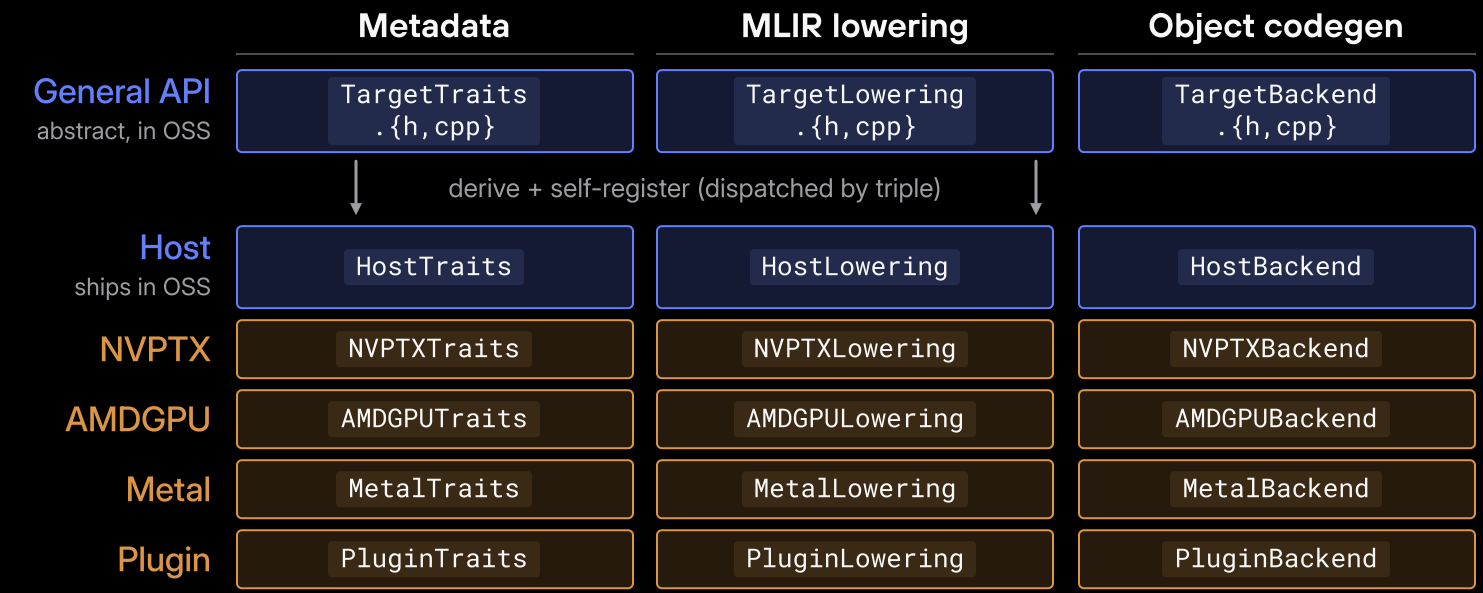
    with out_buf.map_to_host() as host_buf:
        print(TileTensor(host_buf, layout))
```



1. **Slice** device code into its own MLIR module and compile to binary
2. **Merge** compiled offload binary back to host module as final output

Target abstraction — one API set, one impl per accelerator

Three orthogonal layers × one class per backend — shared code calls only the base-class API; each concrete backend subclasses it.



Columns = pipeline layers · rows = accelerators (max_license). Drop a row → that accelerator is gone; drop everything → **host-only OSS** builds.

Self registered backends

- Each target registers itself from its own source files via a static initializer:
  - **Traits** for target-specific metadata, e.g. **name**, file extensions, etc.
  - **Lowering** for target-specific MLIR transforms and lowering.
  - **Backend** for the target-specific LLVM backend.

```
// Mojo/lib/Target/max_license/NVPTX/NVPTXTraits.cpp
static RegisterTargetTraits<NVPTXTraits> registerNVPTXTraits;
// Mojo/KGENToLLVM/Target/max_license/Metal/MetalLowering.cpp
static RegisterTargetLowering<MetalLowering> registerMetalLowering;
// Mojo/Compiler/ObjectCompiler/Target/max_license/MDGPU/MDGPUBackend.cpp
static RegisterTargetBackend<AMDGPUBackend> registerAMDGPUBackend;
```

- TargetBackendRegistry maintains the backends
  - **TargetTraits**, **TargetLowering** and **TargetBackend** pro vide the abstractions/APIs
  - Concrete implementations are dispatched by LLVM triple through a registry lookup

```
const TargetBackend *
TargetBackendRegistry::lookup(const llvm::Triple &triple) const {
  const TargetBackend *directMatch = nullptr;
  for (const std::unique_ptr<TargetBackend> &backend: Backends) {
    const TargetBackend *resolved = backend->resolve(triple);
    if (!resolved)
      continue;
    if (resolved != backend.get())
      return resolved;
    if (!directMatch)
      directMatch = resolved;
  }
  return directMatch;
}
```

- A supported **target** registers a full implementation from its own source file; a **triple** with no registered traits/lowering/backend is an error at the use site.

Backend Plugins

- Built-in Backend vs True Plugin Backend as .so
  - Built-in Backend: Nvidia, AMDGPU, Apple Metal
  - Plugin Backend
    - Independent from core Mojo compiler
    - Register the backend by dlopen through the Plugin shim
    - Dynamic linking to LLVM/MLIR
- Extending Plugin abstract APIs to add more backend specific lowering entrance points.

10:30 – 11:00am	Mojo stdlib plugin & driver plugin	Connor Gray and Manoj Rajagopalan
11:30 – 12:00pm	How Mojo supports non-LLVM back-ends	Kolya Panchenko
2:00 – 2:30pm	Case Study: How Mojo Metaprogramming abstracts away hardware specificity	Kern Handa

Mojo Compiler Architecture Overview

01 Mojo Parser 

02 Mojo Elaborator 

03 Mojo Backend Plugins 

3:00 – 3:30pm Mojo Roadmap  Denis Gurchenkov

